

Lab 5

High Performance Computing
Fall semester 2025-2026

Aikaterini Tsiaousi 3626
Panagiotis Nanousis 3599

January 8, 2026

1 Implementation Details

1.1 Computer Hardware used

The computer that was used to generate the execution times had the characteristics below:

1. **CPU:** Intel(R) Xeon(R) CPU E5-2695 v3 @ 2.30GHz
2. **Memory:** 128GiB Memory
3. **Operating System:** Ubuntu 24.04.3 LTS
4. **Kernel:** Linux 6.8.0-85-generic
5. **Compiler:** NVIDIA (R) Cuda compiler driver

1.2 Methodology

In this project, the main focus is to accelerate an N-body simulation using one or more *GPUs*. The NBody algorithm simulates multiple systems (galaxies) consisting of multiple bodies (planets, atoms, and particles). Each system is independent of each other. However, each body is acted upon by different forces, electrostatic and gravitational. At each time step, for each body in all galaxies, the algorithm calculates the net force, the body's position, velocity, and acceleration. Then, also considering the time passed, it calculates the body's next position into the system/galaxy. Since these computations have to be applied in all bodies of each system, we end up doing multiple computations and iterations. Therefore, to reduce the execution time as much as possible, we parallelize the code first on the CPU, then on the GPU, and add extra optimizations at the end.

1.3 Device Query

```
Device 0: Tesla K80
Compute capability: 3.7
Total global memory: 11441 MB
Shared memory/block: 48 KB
Registers/block: 65536
Warp size: 32
Max threads/block: 1024
Multiprocessors: 13
Max threads dim: (1024, 1024, 64)
Max grid size: (2147483647, 65535, 65535)
```

2 Original Code Analysis

```
void bodyForce(Body * p, float dt, int n) {
    int i, j;
    float Fx, Fy, Fz, dx, dy, dz, distSqr, invDist, invDist3;
    for (i = 0; i < n; i++) {
        Fx = 0.0f;
        Fy = 0.0f;
        Fz = 0.0f;

        for (j = 0; j < n; j++) {
            dx = p[j].x - p[i].x;
            dy = p[j].y - p[i].y;
            dz = p[j].z - p[i].z;
            distSqr = dx * dx + dy * dy + dz * dz + SOFTENING;
            invDist = 1.0f / sqrtf(distSqr);
            invDist3 = invDist * invDist * invDist;

            Fx += dx * invDist3;
            Fy += dy * invDist3;
            Fz += dz * invDist3;
        }
        p[i].vx += dt * Fx;
        p[i].vy += dt * Fy;
        p[i].vz += dt * Fz;
    }
}

void integrate(Body * p, float dt, int n) {
    int i;
    for (i = 0; i < n; i++) {
        p[i].x += p[i].vx * dt;
        p[i].y += p[i].vy * dt;
        p[i].z += p[i].vz * dt;
    }
}
```

Figure 1: The original code for the Nbody functions

```
/* Time-steps */
for (iter = 1; iter <= nIters; iter++) {
    /* Galaxies */
    for (sys = 0; sys < num_systems; sys++) {
        /* Calculate offset for the galaxy */
        system_ptr = &data[sys * bodies_per_system];

        /* Compute forces & integrate for the galaxy */
        bodyForce(system_ptr, dt, bodies_per_system);
        integrate(system_ptr, dt, bodies_per_system);
    }
}
```

Figure 2: The main for loop that calls the above functions and consist of the whole Nbody simulation.

The original code provided for this Project is rather pretty simple and easy to optimize. Spoiler alert, that was not the case. The code consists of two functions, `bodyForce` and `integrate`. The first function updates a single galaxy by calculating the forces applied and the total velocity of each object. The other function updates the position of each body using the velocity calculated before and the time that has passed from the previous time recorded. Combining the functions to make the final algorithm synthesis, there are two for loops, one that represents the time, and the other that goes through all the systems/galaxies. Looking at the code, we can easily observe that there is no dependency between systems, nor is there any dependency between each body that is currently being processed (outer loop of the `bodyForce` function).

To test this program, we used the `galaxy_data.bin` file which has 32 systems containing 8192 bodies each. For the execution of the test, we perform 20 iterations of the algorithm, representing 20 time sections passed. The serial code without any parallelization or optimization has an approximate execution time of 200 s and a total throughput of 0.216 Billion Interactions / second.

3 Parallelization on CPU using OpenMP

Parallelizing the above code by utilizing multiple CPUs is rather easy. As we noted earlier, the code is massively parallel both on the system's loop and outer time's loop of the bodyForce function. Since we only have 28 cores (56 threads), we cannot parallelize both. Hence, we tried both and chose the one with the best performance.

```
void bodyForce(Body * p, float dt, int n) {
    int i, j;
    float Fx, Fy, Fz, dx, dy, dz, distSqr, invDist, invDist3;
    #pragma omp parallel for private(i,j,Fx,Fy,Fz,dx,dy,dz,distSqr,invDist,invDist3)
    for (i = 0; i < n; i++) {
        Fx = 0.0f;
        Fy = 0.0f;
        Fz = 0.0f;

        for (j = 0; j < n; j++) {
            dx = p[j].x - p[i].x;
            dy = p[j].y - p[i].y;
            dz = p[j].z - p[i].z;
            distSqr = dx * dx + dy * dy + dz * dz + SOFTENING;
            // inverse srt goes brrrrrrrr here
            invDist = 1.0f / sqrtf(distSqr);
            invDist3 = invDist * invDist * invDist;

            Fx += dx * invDist3;
            Fy += dy * invDist3;
            Fz += dz * invDist3;
        }
        p[i].vx += dt * Fx;
        p[i].vy += dt * Fy;
        p[i].vz += dt * Fz;
    }
}

void integrate(Body * p, float dt, int n) {
    int i;
    #pragma omp parallel for private(i)
    for (i = 0; i < n; i++) {
        p[i].x += p[i].vx * dt;
        p[i].y += p[i].vy * dt;
        p[i].z += p[i].vz * dt;
    }
}
```

Figure 3: The code for parallelizing the bodyForce and Integration Functions.

```
/* Time-steps */
for (iter = 1; iter <= nIters; iter++) {
    /* Galaxies */
    #pragma omp parallel for private(sys,system_ptr)
    for (sys = 0; sys < num_systems; sys++) {
        /* Calculate offset for the galaxy */
        system_ptr = &data[sys * bodies_per_system];

        /* Compute forces & integrate for the galaxy */
        bodyForce(system_ptr, dt, bodies_per_system);
        integrate(system_ptr, dt, bodies_per_system);
    }
}
```

Figure 4: The code for parallelizing between each system.

Both had a pretty decent speedup of 10×, with the parallelization of the functions slightly faster 9.4 s compared to 11.3 s. Consequently, the final throughput of the CPU code is **4.546 Billion Interactions / second**.

4 Parallelization on GPUs with CUDA

4.1 Initial implementation

From a talk inside the class, we set the goal of reducing the time to roughly 100 *ms*. In order to achieve that, we started by tracking down how to parallelize the kernels so that we get better performance. Our first idea was to parallelize the code with each thread inside a warp working on a single body and then having multiple blocks for each system. This had an execution time of 24 *s*, very far away from both our goal and the CPU performance, so we scrapped that idea (we will get back to it in a later section). Our next thought was to have each thread inside a block to be executed together in order to calculate a single body's new position. This had its own challenge that all the body calculations did not fit inside a block, 8192 threads were needed, but we only had 1024 threads per block at most.



```
__global__ void bodyForceKernel(Body * p, float dt, int n) {
    // splits the tile since we have at most 1024 threads per block
    int inner_tile_id = blockIdx.y;
    int inside_tile_idx = threadIdx.x;
    int i = blockIdx.x;
    int j = inside_tile_idx + inner_tile_id * blockDim.x;
    Body pi = p[i];
    Body pj = p[j];

    __shared__ float total_Fx;
    __shared__ float total_Fy;
    __shared__ float total_Fz;
    // Initiating the Force accumulators
    __shared__ float Fx[CUDA_BLOCK_SIZE];
    __shared__ float Fy[CUDA_BLOCK_SIZE];
    __shared__ float Fz[CUDA_BLOCK_SIZE];
    float dx, dy, dz, distSqr, invDist, invDist3;
    dx = pj.x - pi.x;
    dy = pj.y - pi.y;
    dz = pj.z - pi.z;
    distSqr = dx * dx + dy * dy + dz * dz + SOFTENING;
    invDist = 1.0f / sqrtf(distSqr);

    invDist3 = invDist * invDist * invDist;
    Fx[inside_tile_idx] = dx * invDist3;
    Fy[inside_tile_idx] = dy * invDist3;
    Fz[inside_tile_idx] = dz * invDist3;
    __syncthreads();

    // add the stuff inside each subtile
    for (unsigned int s = CUDA_BLOCK_SIZE/2; s > 0; s >>= 1) {
        if(i==0){
            printf("s=%d, inside_tile_idx=%d, Fx=%f\n", s, inside_tile_idx, Fx[inside_tile_idx]);
        }
        if (inside_tile_idx < s) {
            Fx[inside_tile_idx] += Fx[inside_tile_idx + s];
            Fy[inside_tile_idx] += Fy[inside_tile_idx + s];
            Fz[inside_tile_idx] += Fz[inside_tile_idx + s];
        }
        __syncthreads();
    }
    if(inside_tile_idx == 0){
        atomicAdd(&p[i].vx, dt * Fx[0]);
    }
    if(inside_tile_idx == 1){
        atomicAdd(&p[i].vy, dt * Fy[0]);
    }
    if(inside_tile_idx == 2){
        atomicAdd(&p[i].vz, dt * Fz[0]);
    }
}

__global__ void integrateKernel(Body * p, float dt, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        p[i].x += p[i].vx * dt;
        p[i].y += p[i].vy * dt;
        p[i].z += p[i].vz * dt;
    }
}
```

Grid -> <8192,8> Block -> <1024>

Threads work on groups of 8 to add calculate a single body

Reduction to add forces that were calculated inside the block

Adding the forces as atomic since 8 blocks work on the same body.

Figure 5: The first parallelized version of the code.

As you can observe from the above code, a group of 8 blocks cooperate in order to calculate the new position of a single body. This is a very fine grained parallelism. Furthermore, we use many cycles to execute the atomic additions and the reduction. The final execution time for this code was 16.407 seconds with a throughput of 2.618 Billion Interactions / second, still very slow compared to the CPU code but faster than the serial code at least.

4.2 Further Optimizations

In order to improve the performance of our code, we scrapped the idea of having 4 blocks coordinating in order to calculate the new position of a single body, but instead made each thread work on 8 different interactions between bodies. Along with that, we pushed the performance by utilizing an approximate math function in order to calculate the inverse square root provided by NVIDIA. This function is called *rsqrtf()*, but it turned out that it minorly increases performance. Furthermore, we swapped the expensive calculation of the distance that contained 3 multiplications and 3 additions with a multiply accumulate instruction.

```
__global__ void bodyForceKernel2(Body * p, float dt, int n) {
    // splits the tile since we have at most 1024 threads per block
    int inside_tile_idx = threadIdx.x;
    int i = blockIdx.x;

    float fx = 0.0f;
    float fy = 0.0f;
    float fz = 0.0f;
    __shared__ Force force_shared;
    Body bi = p[i];

    // loop over 8 tiles
    for(int tile = 0; tile < n/blockDim.x; tile++){
        int j = inside_tile_idx + tile * blockDim.x;
        Body bj = p[j];
        float dx, dy, dz, distSqr, invDist, invDist3;

        dx = bj.x - bi.x;
        dy = bj.y - bi.y;
        dz = bj.z - bi.z;
        distSqr = fmaf(dx, dx, fmaf(dy, dy, fmaf(dz, dz, SOFTENING)));
        // distSqr = dx * dx + dy * dy + dz * dz + SOFTENING;
        invDist = rsqrtf(distSqr);
        // invDist = 1.0f / sqrtf(distSqr);
        invDist3 = invDist * invDist * invDist;
        fx += dx * invDist3;
        fy += dy * invDist3;
        fz += dz * invDist3;
    }
    force_shared.x[inside_tile_idx] = fx;
    force_shared.y[inside_tile_idx] = fy;
    force_shared.z[inside_tile_idx] = fz;
    __syncthreads();

    for (unsigned int s = CUDA_BLOCK_SIZE/2; s > 0; s >>= 1) {
        if (inside_tile_idx < s) {
            force_shared.x[inside_tile_idx] += force_shared.x[inside_tile_idx + s];
            force_shared.y[inside_tile_idx] += force_shared.y[inside_tile_idx + s];
            force_shared.z[inside_tile_idx] += force_shared.z[inside_tile_idx + s];
        }
        __syncthreads();
    }
    if(inside_tile_idx == 0){
        atomicAdd(&p[i].vx, dt * force_shared.x[0]);
        atomicAdd(&p[i].vy, dt * force_shared.y[0]);
        atomicAdd(&p[i].vz, dt * force_shared.z[0]);
    }
}
```

Each thread works on 8 interactions

Improved floating point calculations

Figure 6: The improved fine grained parallelized function.

All in all this decreased our total execution time to 9.796 seconds and 4.384 Billion Interactions / second. This matches our Parallelized CPU performance, but we are still very far away from our goal of 100ms.

4.3 Fixing Uncoalesced Memory Accesses

After the previous optimizations, we were stuck in a standstill. We could not find any other way to improve performance. This is why we started using NVIDIA Nsight Compute to see where the bottleneck was. This helped us find out that the memory operations we were doing were uncoalesced and hindered our performance by a lot.

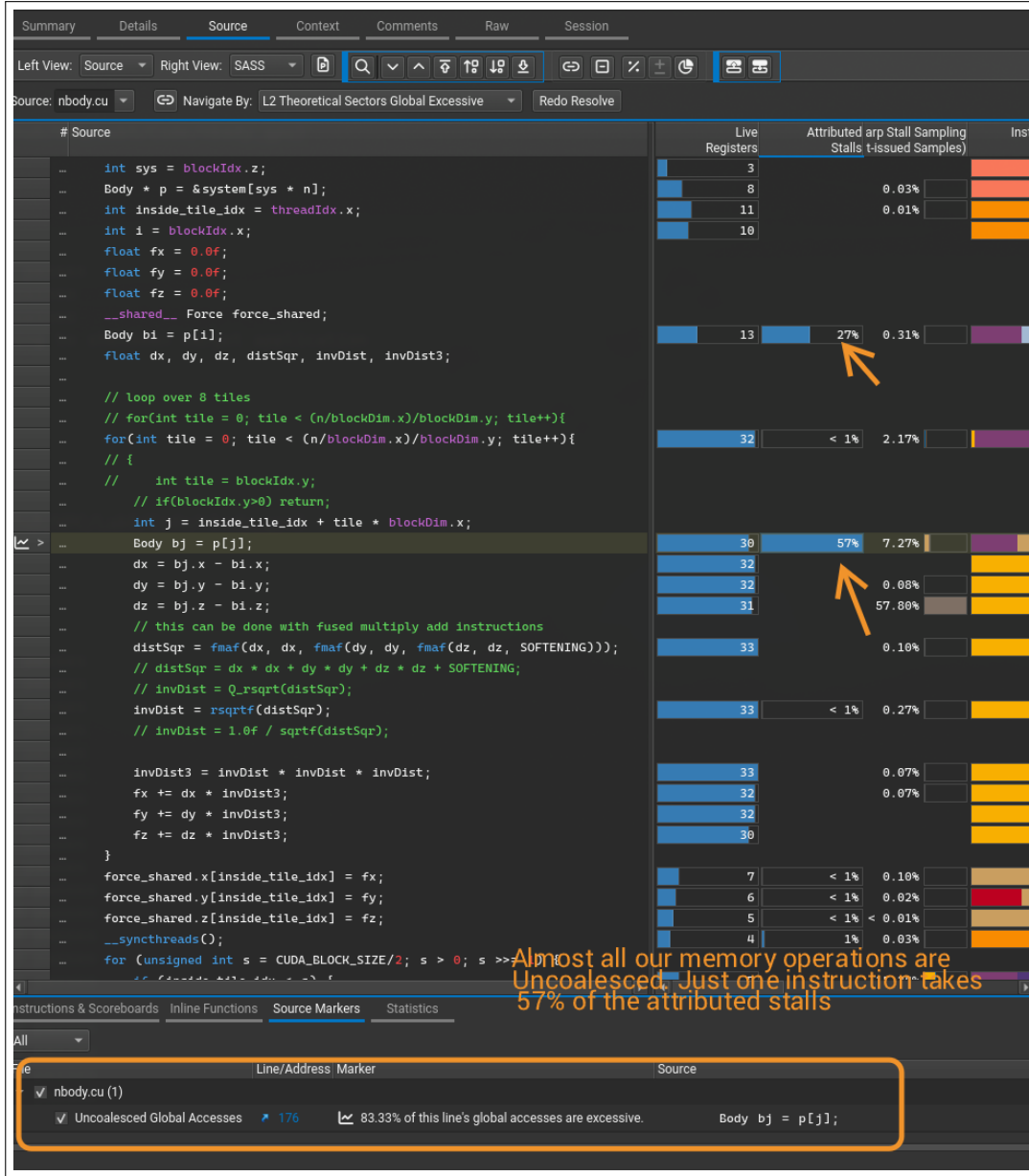


Figure 7: Nvidia Nsight showing us that 83% of our memory accesses are uncoalesced.

This meant that we had to fix the coalesced memory accesses of the Body variable. To do this, we had to switch from loading our body states from an array of structs (Body struct) to a struct of arrays. To make this happen, we had to change the way Body was stored in RAM. Now, the new Body struct contains an array of positions and velocity of bodies.

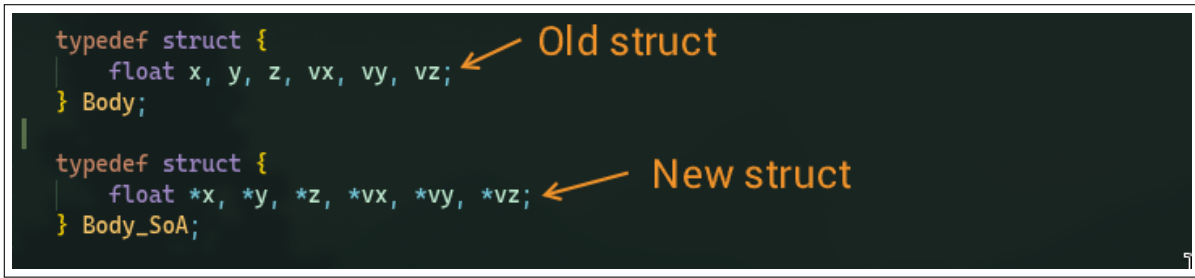


Figure 8: The new Body as a struct of arrays.

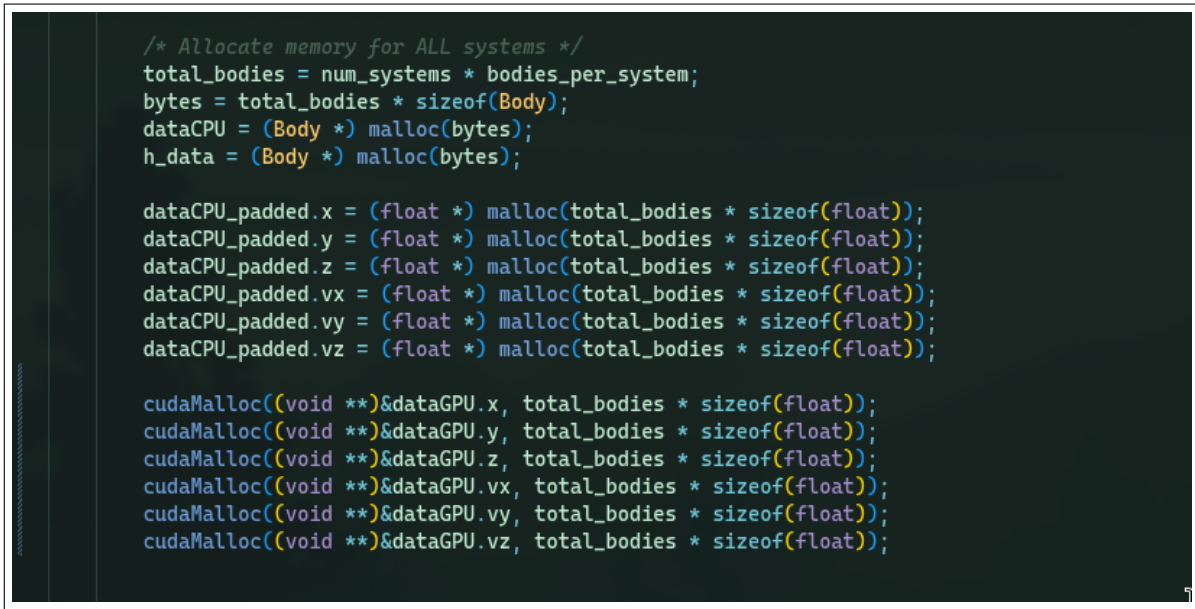


Figure 9: As a struct of arrays we have to allocate memory for each variable inside the struct.

By applying this optimization, we now have uncoalesced memory accesses inside our code. This greatly increases the performance of our code, dropping the execution time of our code to 2.5 s and 17.484 Billion Interactions / second.

4.4 Multiple GPUs

Then, when running with different examples and different number of bodies, we noticed that by simply decreasing the number of systems calculated by the algorithm, we linearly decreased the execution time. This is why we had the idea of using all 4 Tesla K80 GPUs inside the Venus Computer. This should theoretically provide us with a 4× speedup.

Our main thought to implement this was to minorly change our code so that it will parallelize systems as well. From now on, the CPU will no longer call the kernel 32 times, each time the program is executed for each time iteration (20 times). Instead, the CPU will call it once and the GPU will handle all systems.


```

__global__ void bodyForceKernel2(BodySoA system, float dt, int n) {
    // splits the tile since we have at most 1024 threads per block
    int sys = blockIdx.z;
    int pointer_index = sys * n;
    int inside_tile_idx = threadIdx.x;
    int i = blockIdx.x;
    float fx = 0.0f;
    float fy = 0.0f;
    float fz = 0.0f;
    __shared__ Force force_shared;
    Body bi;
    bi.x = system.x[pointer_index + i];
    bi.y = system.y[pointer_index + i];
    bi.z = system.z[pointer_index + i];
    bi.vx = system.vx[pointer_index + i];
    bi.vy = system.vy[pointer_index + i];
    bi.vz = system.vz[pointer_index + i];
    float dx, dy, dz, distSqr, invDist, invDist3;

    // loop over 8 tiles
    for(int tile = 0; tile < (n/blockDim.x)/blockDim.y; tile++){
        int j = inside_tile_idx + tile * blockDim.x;
        Body bj;
        bj.x = system.x[pointer_index + j];
        bj.y = system.y[pointer_index + j];
        bj.z = system.z[pointer_index + j];
        bj.vx = system.vx[pointer_index + j];
        bj.vy = system.vy[pointer_index + j];
        bj.vz = system.vz[pointer_index + j];
        dx = bj.x - bi.x;
        dy = bj.y - bi.y;
        dz = bj.z - bi.z;
        distSqr = fmaf(dx, dx, fmaf(dy, dy, fmaf(dz, dz, SOFTENING)));
        invDist = rsqrtf(distSqr);
        invDist3 = invDist * invDist * invDist;
        fx += dx * invDist3;
        fy += dy * invDist3;
        fz += dz * invDist3;
    }
    force_shared.x[inside_tile_idx] = fx;
    force_shared.y[inside_tile_idx] = fy;
    force_shared.z[inside_tile_idx] = fz;
    __syncthreads();
    for (unsigned int s = CUDA_BLOCK_SIZE/2; s > 0; s >>= 1) {
        if (inside_tile_idx < s) {
            force_shared.x[inside_tile_idx] += force_shared.x[inside_tile_idx + s];
            force_shared.y[inside_tile_idx] += force_shared.y[inside_tile_idx + s];
            force_shared.z[inside_tile_idx] += force_shared.z[inside_tile_idx + s];
        }
        __syncthreads();
    }
    if(inside_tile_idx == 0){
        system.vx[pointer_index + i] += dt * force_shared.x[0];
        system.vy[pointer_index + i] += dt * force_shared.y[0];
        system.vz[pointer_index + i] += dt * force_shared.z[0];
    }
}

```

Dimension Z gives us which system we are working on

We calculate the pointer to the system on our own instead of relying on the CPU to do it for us

Figure 10: The kernel changed to handle multiple Systems inside a single kernel.

In that way, we can instead split the GPUs to execute 32/4 (8) systems each. To implement this, we first have to get the data inside all our GPUs. This is done by setting the device and then doing a cudaMalloc for that specific device as shown in the figure below.

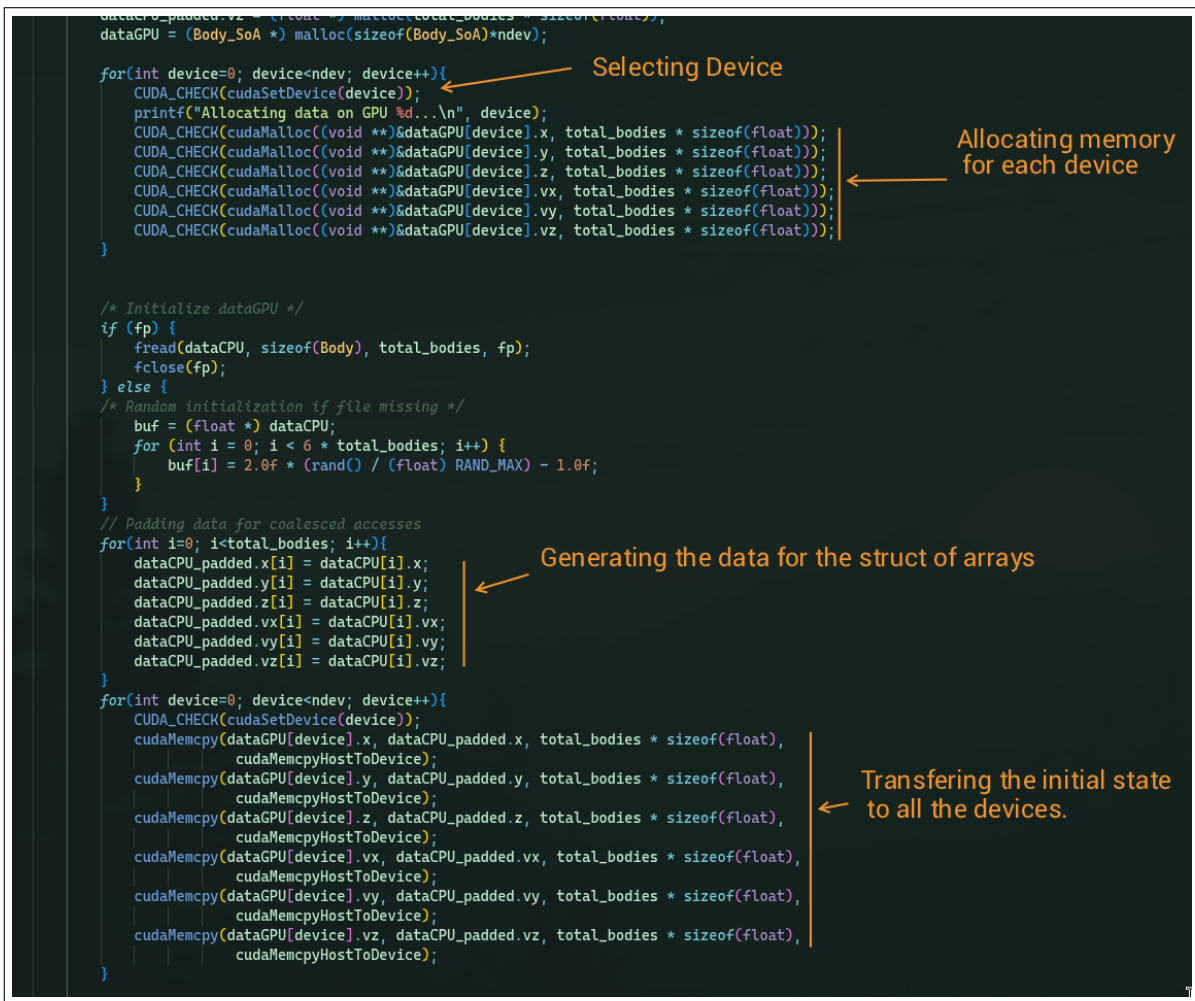


Figure 11: Allocation of memory for all the devices

A minor modification to our kernel code needs to be made in order to take into consideration the current device on which this is being executed. To implement it, we multiply the device ID by the total number of systems that each device works on.

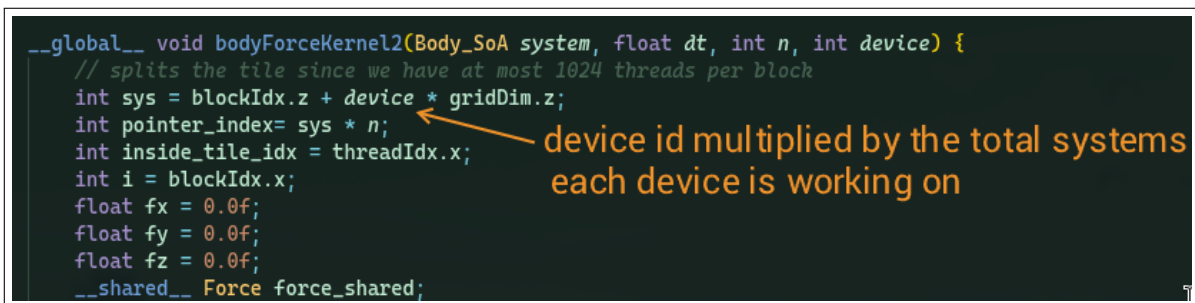


Figure 12: Minor tweak on our kernel to allow for multiple GPUs

```

__global__ void integrateKernel_SoA_all(Body_SoA p, float dt, int n, int device) {
    int sys = blockIdx.y + device * gridDim.y;

    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int pointer_index = sys * n ;
    if (i < n) {
        p.x[pointer_index + i] += p.vx[pointer_index + i] * dt;
        p.y[pointer_index + i] += p.vy[pointer_index + i] * dt;
        p.z[pointer_index + i] += p.vz[pointer_index + i] * dt;
    }
}

```

Figure 13: The integration kernel changed to support multiple concurrent systems and devices.

Our next step is to launch the kernels and make them run in parallel, so to do that we are going to use streams. Hence a new stream is going to be needed for each device we are working on. After everything above is completed, we can launch all our kernels on our devices.

```

dim3 Block(bodies_per_system, 1, num_systems/ndev);
dim3 Block_Integrate(8, num_systems/ndev);

cudaStream_t *streams = (cudaStream_t *) malloc(ndev * sizeof(cudaStream_t));
for (int i = 0; i < ndev; i++) {
    CUDA_CHECK(cudaSetDevice(i));
    CUDA_CHECK(cudaStreamCreate(&streams[i]));
}

// Time-steps
for (iter = 1; iter <= nIters; iter++) {
    /* Galaxies */
    tempTime = GetTimer();
    for (int device=0; device<ndev; device++){
        CUDA_CHECK(cudaSetDevice(device));
        bodyForceKernel2<<<Block, CUDA_BLOCK_SIZE, 0, streams[device]>>>(dataGPU[device], dt, bodies_per_system, device);
    }

    for (int device = 0; device < ndev; device++) {
        CUDA_CHECK(cudaSetDevice(device));
        CUDA_CHECK(cudaStreamSynchronize(streams[device]));
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaPeekAtLastError());
    }

    bodyforceTime += GetTimer() - tempTime;
    tempTime = GetTimer();
    for (int device=0; device<ndev; device++){
        CUDA_CHECK(cudaSetDevice(device));
        integrateKernel_SoA_all<<<Block_Integrate, CUDA_BLOCK_SIZE>>>(dataGPU[device], dt, bodies_per_system, device);
    }

    for (int device = 0; device < ndev; device++) {
        CUDA_CHECK(cudaSetDevice(device));
        CUDA_CHECK(cudaStreamSynchronize(streams[device]));
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaPeekAtLastError());
    }

    integrateTime += GetTimer() - tempTime;
}

```

Stream creation for each device.

Launching body kernel for each device

Wait for all kernels to finish

Launching integration kernel for each device

Wait for the kernel to finish before going to next iteration

Figure 14: Stream creation and kernel launch for multiple GPUs

Finally, we have to transfer the data back to the GPUs. This is a bit more tricky than simply copying the full data from the output to our data buffer. It is worth noting that what we did earlier, allocating the full total bodies per GPU is not very efficient, but since we have enough Memory, we left it like that for simplicity.

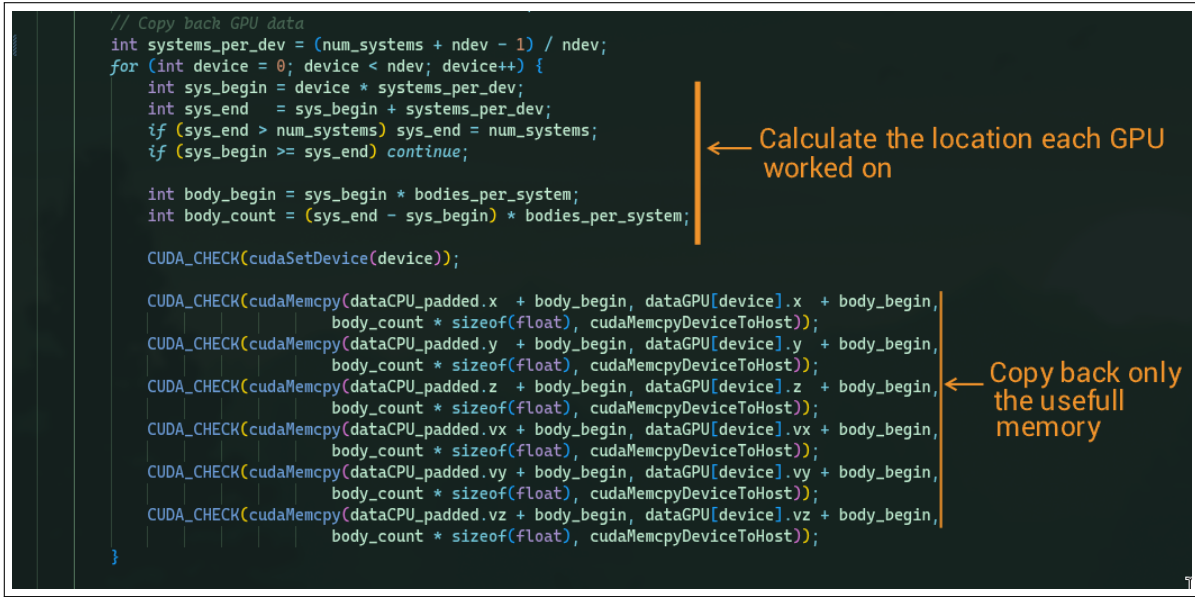


Figure 15: Transferring back the data from multiple GPUs to our buffer for verifying the correctness.

After applying all the changes to our code and running the example, we ended up having, as expected, a $4\times$ speedup when using multiple GPUs. The total execution time was 0.580300 s ($\text{std} \pm 0.000949\text{ s}$) with 73.996 BIPS ($\text{std} \pm 0.134\text{ BI/s}$). With all these changes, we got very close to our final goal, but still did not reach it, so we continue to gain more performance.

4.5 Fixing Occupancy

A further problem that was highlighted by NVIDIA Nsight is occupancy. According to the Nsight analysis, we used only 50% of the theoretical occupancy 66%, which is possible to achieve on our GPU. This was caused because the threads did not take a lot of time to finish their execution, and consequently there were no warps to swap to when one warp was stalling, waiting for memory access. Fortunately, this was easily fixed by reducing the block size from 1024 to 256. After trying and testing different block sizes, only 256 seemed to be the most stable one. This change further increased our performance to 109.634 BIPS ($\text{std} \pm 2.230\text{ BI/s}$) and with an execution time of 0.3919 s ($\text{std} \pm 0.008359$).

4.6 Going from fine grained to coarse grained parallelism

After applying the optimizations made above, we were stuck and could not really think of any further way to improve performance. NVIDIA Nsight indicated that our remaining bottlenecks were memory accesses along with the reduction of each block to get the final velocity. This is why we revisited the previous idea of each thread working on a single body and parallelizing per body and per system. Although this approach did not work previously, probably due to the coalesced memory accesses, to our surprise, switching to coarse grained parallelism not only was not detrimental but actually it outperformed our fine tuned body function kernel. The code was also much simpler and easily understandable. This is why we dived deeper into optimizing this version of the code.

```

__global__ void bodyForceKernel_coarse(Body_SoA system, float dt, int n, int device) {
    // splits the tile since we have at most 1024 threads per block
    int sys = blockIdx.z + device * gridDim.z;
    int pointer_index = sys * n;
    int body_id = threadIdx.x + blockIdx.x * blockDim.x;
    if (body_id >= n) return;
    float fx = 0.0f;
    float fy = 0.0f;
    float fz = 0.0f;
    float dx, dy, dz, distSqr, invDist, invDist3;
    for(int j=0; j<n; j++){ ← Each thread working on a single body
        dx = system.x[pointer_index + j] - system.x[pointer_index + body_id];
        dy = system.y[pointer_index + j] - system.y[pointer_index + body_id];
        dz = system.z[pointer_index + j] - system.z[pointer_index + body_id];
        distSqr = fmaf(dx, dx, fmaf(dy, dy, fmaf(dz, dz, SOFTENING)));
        invDist = rsqrtf(distSqr);
        invDist3 = invDist * invDist * invDist;
        fx = fmaf(dx, invDist3, fx);
        fy = fmaf(dy, invDist3, fy);
        fz = fmaf(dz, invDist3, fz);
    }
    system.vx[pointer_index + body_id] += dt * fx;
    system.vy[pointer_index + body_id] += dt * fy;
    system.vz[pointer_index + body_id] += dt * fz;
}

```

Figure 16: The kernel code for coarse grained parallelization.

The performance gained from this code was impressive. It also demonstrated that we should have started by mimicking the parallelization of our OpenMP code instead of trying to get even finer grained parallelism. The execution time decreased to 0.2722 s (std±0.001751s) having a throughput of 157.762BI/s (std±0.901BI/s).

4.7 Loop unrolling

Another optimization that we had tried earlier but was not very helpful was to unroll the loop of iterations. For this much simpler code, unrolling the inner loop was beneficial. It seemed to provide a performance boost of around 10%, reducing our execution time to 0.258300 (std±0.016997s).

```

#pragma unroll 8
for(int j=0; j<n; j++){
    dx = system.x[pointer_index + j] - system.x[pointer_index + body_id];
    dy = system.y[pointer_index + j] - system.y[pointer_index + body_id];
    dz = system.z[pointer_index + j] - system.z[pointer_index + body_id];
    distSqr = fmaf(dx, dx, fmaf(dy, dy, fmaf(dz, dz, SOFTENING)));
    invDist = rsqrtf(distSqr);
    invDist3 = invDist * invDist * invDist;
    fx = fmaf(dx, invDist3, fx);
    fy = fmaf(dy, invDist3, fy);
    fz = fmaf(dz, invDist3, fz);
}

```

Figure 17: The unroll pragma that unrolls the loop on GCC

4.8 Memory Tiling

Throughout our analysis, NVIDIA Nsight kept showing that our bottleneck was the memory accesses inside the loop. We also noticed that we reuse most of the memory loads across threads of the same block. Ideally, each thread would load a single body and cache it into shared memory for all other threads to use. Unfortunately, the shared memory is not big enough to accommodate three arrays of 8192 floats, and instead we had to split it into tiles. Furthermore, consider that we have 256 threads per block. This means that we could cache 256 bodies, work on them, and then go onto the next set of 256 bodies. In that way, we would avoid reissuing the same memory accesses and wasting memory cycles.

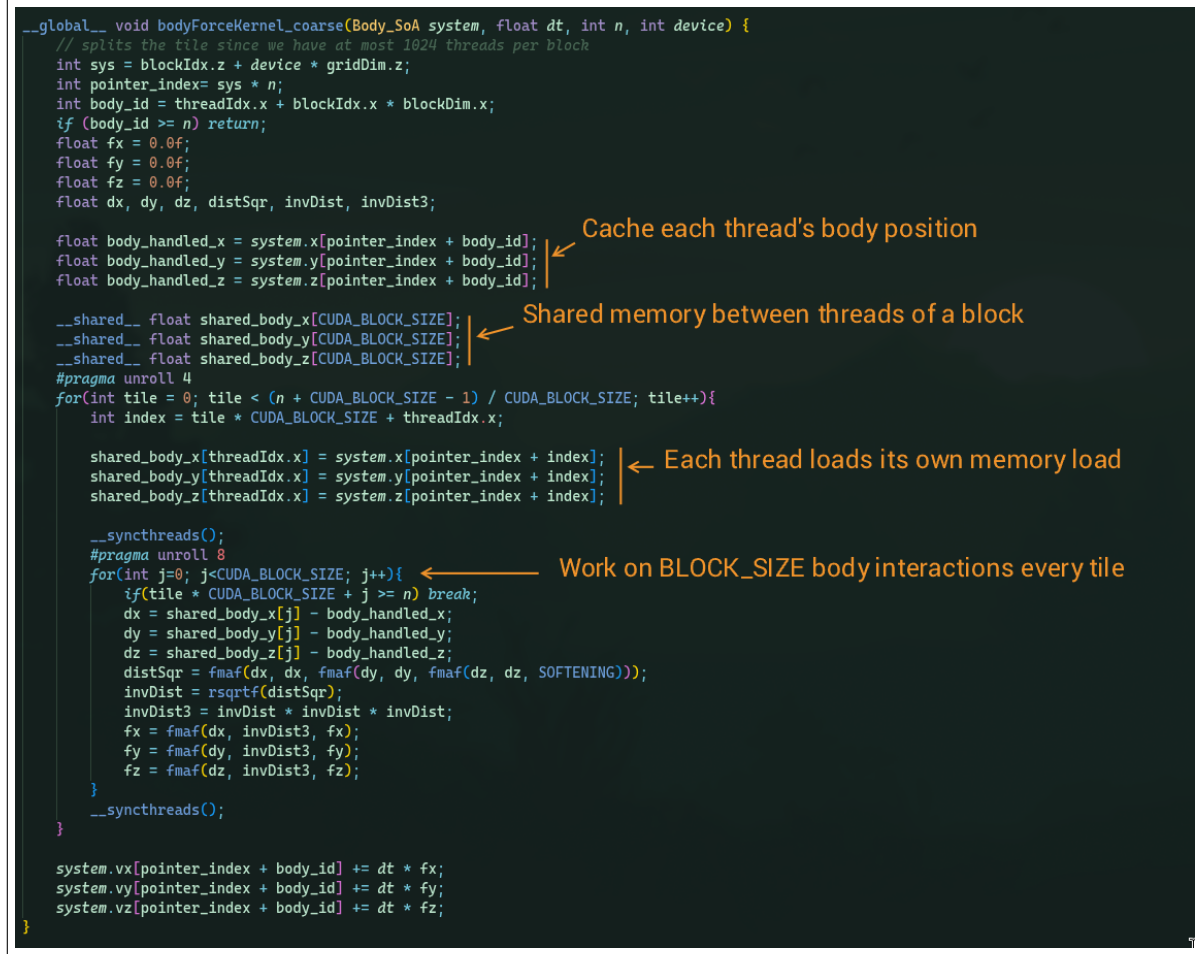


Figure 18: Tiling of the algorithm in order to utilize the throughput of the shared memory

After applying and testing the memory tiling optimizations, that actually came up with a very big effect on performance, more than halving the execution time of the program. Specifically, the execution time ended up being 0.101500 s ($\text{std} \pm 0.000527$) with a throughput of 424.169BI/s ($\text{std} \pm 2.060\text{BI/s}$). Finally after all these different implementations and optimizations, we finally managed to reach our initial goal of 100ms execution time of our kernel.

Table 1: Performance Summary with Execution Time, Speedup, Throughput, and Standard Deviation

Optimization Step	Time (s)	Speedup	Throughput (BI/s)	Std. Dev.
Serial CPU	200.000	1.00×	0.216	—
OpenMP CPU	9.400	21.28×	4.546	—
GPU v0 (warp/body, scrapped)	24.000	8.33×	—	—
GPU fine-grained v1	16.407	12.19×	2.618	—
GPU fine-grained v2 (rsqrtf + FMA)	9.796	20.42×	4.384	—
SoA (coalesced memory)	2.500	80.00×	17.484	—
Multi-GPU (4× K80)	0.580300	344.58×	73.996	±0.134 BI/s
Occupancy fix (block = 256)	0.391900	510.26×	109.634	±2.230 BI/s
Coarse-grained GPU	0.272200	734.72×	157.762	±0.901 BI/s
Loop unrolling	0.258300	774.19×	166.250	±0.016997 s
Shared-memory tiling	0.101500	1970.54×	424.169	±2.060 BI/s

5 Correctness Verification

5.1 Verification strategy

In order to ensure that our program is not only fast and optimized, but also correct, we implemented a verification process. Our initial thought was that N-Body simulation is sensitive to floating-point optimizations, so we do not expect exactly the same results but an approximate similarity, having a logical elasticity of 0.001. To validate our results, we run our two different implementations, one with the CPU OpenMP version and the other with the GPU final version. After running both, we insert the final positions of bodies of all the systems in two different arrays and compare these position arrays for every single body by subtracting the two values of the two arrays. If the result is $\leq \pm 0.001$, within the acceptable limits, we consider the two positions as the same, confirming the validity of our GPU kernels.

5.2 Arbitrary size support

Furthermore, to satisfy the requirement for having random dataset sizes, our CUDA kernels were changed to make sure that for arbitrary large body count, we would both work on the extra bodies but also not corrupt the memory of the other systems.


```

__global__ void bodyForceKernel_coarse(Body_SoA system, float dt, int n, int device) {
    // splits the tile since we have at most 1024 threads per block
    int sys = blockIdx.z + device * gridDim.z;
    int pointer_index = sys * n;
    int body_id = threadIdx.x + blockIdx.x * blockDim.x;
    float fx = 0.0f;
    float fy = 0.0f;
    float fz = 0.0f;
    float dx, dy, dz, distSqr, invDist, invDist3;

    float body_handled_x=0.0f;
    float body_handled_y=0.0f;
    float body_handled_z=0.0f;

    if (body_id < n) {
        body_handled_x = system.x[pointer_index + body_id];
        body_handled_y = system.y[pointer_index + body_id];
        body_handled_z = system.z[pointer_index + body_id];
    }

    __shared__ float shared_body_x[CUDA_BLOCK_SIZE];
    __shared__ float shared_body_y[CUDA_BLOCK_SIZE];
    __shared__ float shared_body_z[CUDA_BLOCK_SIZE];
    #pragma unroll 4
    for(int tile = 0; tile < (n + blockDim.x - 1) / blockDim.x; tile++){
        int index = tile * blockDim.x + threadIdx.x;
        if(index < n) {
            shared_body_x[threadIdx.x] = system.x[pointer_index + index];
            shared_body_y[threadIdx.x] = system.y[pointer_index + index];
            shared_body_z[threadIdx.x] = system.z[pointer_index + index];
        }
        __syncthreads();
        #pragma unroll 8
        for(int j=0; j<blockDim.x; j++){
            if(tile * blockDim.x + j >= n){
                break;
            }
            dx = shared_body_x[j] - body_handled_x;
            dy = shared_body_y[j] - body_handled_y;
            dz = shared_body_z[j] - body_handled_z;
            distSqr = fmaf(dx, dx, fmaf(dy, dy, fmaf(dz, dz, SOFTENING)));
            invDist = rsqrtf(distSqr);
            invDist3 = invDist * invDist * invDist;
            fx = fmaf(dx, invDist3, fx);
            fy = fmaf(dy, invDist3, fy);
            fz = fmaf(dz, invDist3, fz);
        }
        __syncthreads();
    }

    if(body_id < n){
        system.vx[pointer_index + body_id] += dt * fx;
        system.vy[pointer_index + body_id] += dt * fy;
        system.vz[pointer_index + body_id] += dt * fz;
    }
}

```

Figure 19: Checks for arbitrary number of bodies on the nbody kernel

```

__global__ void integrateKernel_SoA_all(Body_SoA p, float dt, int n, int device) {
    int sys = blockIdx.y + device * gridDim.y;

    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int pointer_index = sys * n;
    if (i < n) {
        p.x[pointer_index + i] += p.vx[pointer_index + i] * dt;
        p.y[pointer_index + i] += p.vy[pointer_index + i] * dt;
        p.z[pointer_index + i] += p.vz[pointer_index + i] * dt;
    }
}

```

Figure 20: Checks for arbitrary number of bodies on the integrate kernel

Hence this works for any number of systems and bodies. The only exception is if the number of bodies is less than the CUDA_BLOCK_SIZE, in which case we can just decrease that as well. It is worth noting that when using the provided galaxy data, we get a maximum difference of 0.007076 compared to the CPU where as when using the randomly generated one, we get a maximum difference of 0.0097 which can be attributed to the different floating point operations on bigger floating points.

6 Performance Evaluation

In order to accurately verify whether it is actually necessary to go through all that process and implement the N-body simulation on *GPUs*, we calculated the time to copy all the memory from the initial implementation using *CPU*, to the final one with *GPUs* and back. The measurements were conducted on the given dataset (32 systems, 8192 bodies per system). The table below summarizes the average execution times.

Metric	Average Time (s)
Total CPU Execution	10.398
Total Memory Allocation time	$0.126 * 4 = 0.504$
Total Kernel Time	0.1015
Total Memory Transfer Time	0.008

It is clearly observable that the total time spent on data transfer between *CPU* and *GPU* is approximately 0.008s. Taking into consideration that the execution time is approximately 0.100s, the time needed for data transfer is considered negligible. So, it does not have a significant impact on execution time. In our opinion, the implementation of *GPU* is necessary. However, we noticed that memory allocation takes quite a long time (0.500s – 0.600s on average) compared to actual computation. However, this is an initialization cost that is paid only once at the start of the program and cannot be changed. It is also worth noting that this can be cut down to a fourth if we managed to allocate the GPU memory in parallel, where as now this is done serially due to the blocking nature of the CudaMalloc. Some further optimizations could have been made to only allocate 1/4th of the memory needed but due to the pressure of time that was never implemented.

7 Conclusion

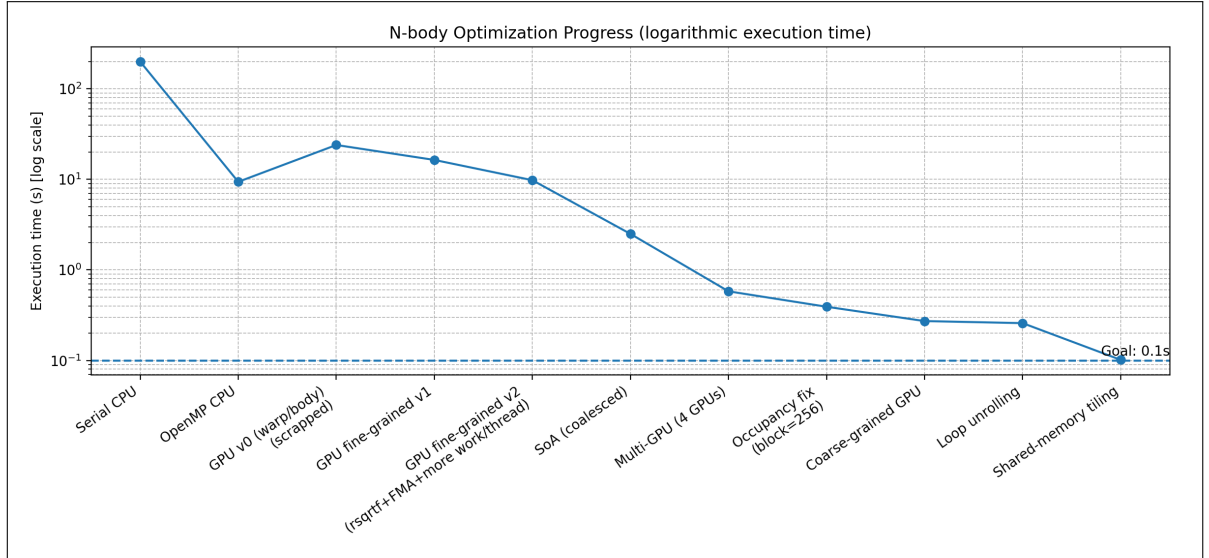


Figure 21: Graph showcasing the progress throughout our optimizations with the final version achieving 101ms of execution time

In this lab, we optimized an N-body simulation by addressing the main performance bottlenecks and progressively introducing parallelism on CPUs and GPUs. OpenMP parallelization achieved an initial speedup of approximately 10× over the serial implementation, which was already very good. Early CUDA implementations using fine-grained parallelism did not perform well due to high synchronization overhead, uncoalesced memory accesses, and low occupancy. By restructuring the data layout from an array-of-structures to a structure-of-arrays, memory coalescing was significantly improved, leading to

large performance gains. Additional optimizations, including approximate math intrinsics, improved occupancy, and multi-GPU execution using CUDA Streams, further reduced execution time and scaled nearly linearly across devices.

Returning to a coarse-grained parallelization strategy proved to be more effective once memory access patterns were optimized. This approach reduced kernel complexity and overhead while better matching the GPU execution model. Finally, tiling body data into shared memory minimized redundant global memory accesses and enabled efficient reuse across threads.

With these optimizations, the final implementation achieved an execution time of approximately **100 ms** and a throughput of **424 Billion Interactions Per Second**, exceeding our initial performance goals while maintaining simulation correctness.